

Trabajo Práctico Integrador

Food Store - Sistema de Gestión de Pedidos de Comida

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a
Distancia

Materia: Programación II

Estudiante: Valentina Lucia Caro

Legajo: 18314

Fecha de entrega: Junio de 2026

Índice

Marco teórico.....	2
1.1 Tecnologías usadas.....	2
2.1 Conceptos clave aplicados.....	2
Decisiones Técnicas y Arquitectura.....	3
2.1 Organización de Paquetes.....	3
2.2 Modelado de programa.....	3
2.3 Demostración de funcionamiento.....	4
Dificultades y soluciones.....	5
3.1 Complejidad en la Implementación de Validaciones.....	5
3.2 Solución.....	6
Bibliografía.....	6
5.1 Enlaces Obligatorios de la Entrega.....	6
5.2 Bibliografía y Webgrafía Utilizada.....	6

Marco teórico

El desarrollo de la aplicación Food Store se fundamentó en la aplicación práctica de conceptos avanzados de Programación Orientada a Objetos (POO), utilizando herramientas del ecosistema Java.

1.1 Tecnologías usadas

El programa fue realizado mediante el entorno de desarrollo integrado NetBeans en lenguaje Java, utilizando la versión Java 21 para aprovechar las optimizaciones de rendimiento y las estructuras modernas (como los bloques switch mejorados utilizando pattern matching).

Se mantuvo una arquitectura de árbol por capas para mantener una estructura limpia y modularizada de paquetes y clases, organizada de la siguiente manera:

```
None
Caro_Valentina_TPI/
├── src/
│   ├── Main/
│   ├── entities/
│   ├── enums/
│   ├── exception/
│   ├── interfaces/
│   ├── services/
│   └── utilities/
└── README.md
```

Bajo esta estrategia, la capa Main quedó limitada exclusivamente a la interacción inicial con el usuario, evitando incorporar procesos complejos dentro de ella.

2.1 Conceptos clave aplicados

Encapsulamiento: Se aplicó de manera estricta al restringir el acceso directo a las variables de las entidades mediante el modificador `private`. El estado interno de los objetos sólo puede ser leído o alterado de forma controlada a través de getters y setters. Estos métodos actúan como una barrera defensiva, interactuando directamente con la clase utilitaria Validador para ejecutar controles sintácticos (bloqueo de datos nulos o cadenas vacías) antes de consolidar cualquier asignación en memoria.

Herencia y Abstracción: Se modeló la clase abstracta Base, la cual no permite ser instanciada por sí misma y define un molde estructural para el dominio. Mediante la herencia, esta clase transfiere de forma limpia los atributos id, eliminado y createdAt a las entidades del sistema (Usuario, Producto, etc.). Esto evita la duplicación de código redundante y establece una base unificada para el manejo de registros.

Polimorfismo: Se evidencia en el sistema bajo dos enfoques fundamentales:

- Sobrescritura de métodos (Override): Implementada en el método abstracto toString() heredado de Base. Cada clase hija de Base cuenta con su propio método toString(), para garantizar que cada uno muestre la información correspondiente de cada entidad.
- Polimorfismo por interfaz: A través de la interfaz Calculable y su método calcularTotal(), implementado de forma obligatoria en la clase Pedido para estructurar su propia lógica del cálculo financiero del negocio.

Genéricos: Introducidos en la clase Validador mediante el método parametrizado <T extends Base> T buscarPorId(ArrayList<T> lista, Long id, ...) . Al utilizar tipos genéricos se logró construir un motor de búsqueda único capaz de operar con cualquier entidad del catálogo (ya sea un Producto, una Categoría o un Usuario) manteniendo seguridad de tipado en tiempo de compilación, eliminando la necesidad de realizar casteos forzados u operaciones repetitivas de búsqueda para cada colección del sistema.

Decisiones Técnicas y Arquitectura

2.1 Organización de Paquetes

El proyecto se estructuró de forma modular siguiendo las pautas recomendadas:

- Main: Contiene la clase Menu.java, encargada exclusiva de la interacción directa con el usuario mediante Scanner.
- entities: Alberga las clases del modelo de dominio (Usuario, Pedido, DetallePedido, Producto, Categoría) que extienden de Base.
- services: Capa de lógica pura que procesa las colecciones en memoria e implementa los casos de uso (CRUD).
- enums: Define las constantes fuertemente tipadas del sistema (Rol, Estado, FormaPago).
- exception: Reúne las excepciones personalizadas del negocio.
- utilities: Aloja la clase estática Validador, responsable de los controles de tipos y búsquedas genéricas.

2.2 Modelado de programa

El desarrollo del sistema se llevó a cabo respetando las asociaciones y restricciones establecidas en el diagrama UML, garantizando una correcta representación del dominio y una adecuada separación de responsabilidades mediante la arquitectura por capas.

En primer lugar, la relación de composición entre las clases Pedido y DetallePedido se implementó de manera que los detalles dependen completamente del ciclo de vida del pedido. Para ello, se incorporó el método obligatorio addDetallePedido(), para agregar los elementos asociados y calcular automáticamente el subtotal correspondiente. De esta forma se mantiene la integridad estructural y se evitan inconsistencias. Además se aseguró la integridad bidireccional entre las entidades relacionadas mediante métodos que fueron diseñados para mantener automáticamente la consistencia de ambos lados de las asociaciones 1..N.

Por otro lado, se aplicó el patrón de arquitectura por capas con el objetivo de centralizar la lógica de negocio y mantener una adecuada separación de responsabilidades. El componente Menu se encarga únicamente de recolectar los datos ingresados y derivarlos al método correspondiente dentro de la capa Service, y luego los servicios ejecutan las validaciones y reglas de negocio necesarias.

2.3 Demostración de funcionamiento

Menú principal:

```
run:

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción:
```

Creación de categoría:

```
Seleccione una opción: 1

--- GESTION DE CATEGORIAS ---
1. Listar categorías
2. Crear categoría
3. Editar categoría
4. Eliminar categoría
0. Volver al Menu Principal
Seleccione una opción: 2
Ingrese nombre de la categoría: Comida rapida
Ingrese descripción: alimentos preparados para comer en el momento
Categoría creada con éxito. ID asignado: 1
```

Manejo de excepciones:

```
--- GESTION DE USUARIOS ---
1. Listar usuarios
2. Crear usuario
3. Editar usuario
4. Eliminar usuario
0. Volver al Menu Principal
Seleccione una opción: 2
Nombre: juan
Apellido: perez
Email: mperez@gmail
Celular: 235561
Contraseña: CDA45
Seleccione Rol (1. ADMIN / 2. USUARIO):
-> 2
Error: El correo electrónico fue registrado por otro usuario.
```

mostrar productos:

```
--- GESTION DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver al Menu Principal
Seleccione una opcion: 1

--- OPCIONES DE LISTADO ---
1. Listado general
2. Filtrar por categoria
Seleccione una opcion: 1
Producto [ID: 3 | Nombre: pizza simple | Precio: $150,00 | Stock: 50 | Disponible: true | Categoria: comida rapida]
```

soft delete:

```
--- GESTION DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver al Menu Principal
Seleccione una opcion: 4
ID del producto a eliminar: 3
Confirmar baja? (S/N): s
Producto retirado

--- GESTION DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver al Menu Principal
Seleccione una opcion: 1

--- OPCIONES DE LISTADO ---
1. Listado general
2. Filtrar por categoria
Seleccione una opcion: 2
Ingrese ID de la categoria: 2
No se encontraron productos para mostrar.
```

Dificultades y soluciones

Durante el desarrollo del sistema Food Store, se presentaron diversos desafíos técnicos asociados al cumplimiento de los requerimientos y a la búsqueda de un programa robusto. A continuación, se detallan los principales obstáculos encontrados y las estrategias implementadas para su resolución:

3.1 Complejidad en la Implementación de Validaciones

El principal desafío no fue la complejidad de cada regla, sino en la necesidad de construir un sistema robusto capaz de validar simultáneamente múltiples niveles. El programa debía controlar la integridad de las entradas por consola (evitando el ingreso de datos nulos o cadenas vacías), las restricciones de tipos primitivos (String, int, Double) para prevenir fallos en el escáner, y el cumplimiento estricto de las reglas lógicas del negocio (como el control de unicidad de emails o la disponibilidad de stock).

Coordinar todas estas capas de validación en cada flujo del menú sin generar código duplicado complejizó el desarrollo.

3.2 Solución

Para solucionarlo se optó por juntar las validaciones de datos en la clase utilitaria Validador, y se implementó el paquete exceptions con una jerarquía estructurada de excepciones propias. Se creó la clase abstracta base NegocioException (heredera de RuntimeException), de la cual derivan todas las excepciones específicas de las reglas lógicas del sistema (como StockInvalidoException o EmailDuplicadoException).

Gracias a este enfoque basado en el polimorfismo, no fue necesario declarar bloques catch para cada subtipo de error en la capa Menu. En su lugar, el menú captura de forma genérica a NegocioException, delegando a Java la tarea de identificar qué subclase disparó el evento en tiempo de ejecución para mostrar su mensaje personalizado.

Bibliografía

5.1 Enlaces Obligatorios de la Entrega

Video explicativo: [Programación II Trabajo Práctico Integrador | UTN TUPAD](#)

Repositorio público: [GitHub/Final-Project](#)

5.2 Bibliografía y Webgrafía Utilizada

1. Oracle. (s. f.). *Java documentation*. Oracle. <https://docs.oracle.com/java/>